

React Deep Dive: Hooks, Rules, Reconciliation, Performance, React DOM

This file is intentionally aligned with the modern react.dev mental model, not just the classic "most common hooks" list. For interviews, that matters because senior frontend and full-stack rounds increasingly test whether you understand:

- render vs commit,
- purity and state snapshots,
- when you do not need an Effect,
- concurrent rendering primitives,
- server/client boundaries,
- external store integration,
- and the newer React 19-era APIs.

1) React's Core Mental Model

1.1 UI as a Function of State

React components are pure descriptions of UI for a given input:

- props,
- state,
- context.

Interview line:

- A React component should behave like a pure function during render: same inputs, same JSX output, with no side effects during rendering.

```
function Greeting({ name }) {  
  return <h1>Hello, {name}</h1>;  
}
```

1.2 Render Phase vs Commit Phase

React work is broadly split into:

1. Render phase: React calculates what the next UI should look like.
2. Commit phase: React applies the minimal DOM mutations.

Important interview points:

- Rendering can be interrupted, restarted, or discarded in concurrent React.
- Side effects must not happen during render because render is not guaranteed to commit.
- DOM writes, subscriptions, timers, analytics, and network synchronization belong in Effects, event handlers, or server actions, not in render.

1.3 State Is a Snapshot

State values are snapshots from a specific render, not mutable variables that update in place.

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Why this matters:

- both updates read the same `count` snapshot,
- so the final result becomes `count + 1` , not `count + 2` .

Correct form when the next state depends on the previous state:

```
setCount((current) => current + 1);
setCount((current) => current + 1);
```

1.4 Components and Hooks Must Be Pure

This is a central react.dev rule.

Do:

- derive values from props/state in render,
- keep render deterministic,
- move side effects to Effects or event handlers.

Do not:

- mutate props,
- mutate external state during render,
- start subscriptions or fetches directly in render for normal client components.

Bad:

```
let totalRenders = 0;

function Widget() {
  totalRenders += 1;
  return <div>{totalRenders}</div>;
}
```

This leaks side effects into render and becomes unreliable in Strict Mode or concurrent rendering.

2) Rules of React and Rules of Hooks

2.1 Rules of Hooks

Hooks must be called:

- at the top level of a component,
- at the top level of a custom Hook,
- in the same order on every render.

Do not call Hooks:

- inside loops,
- inside conditionals,
- inside nested functions,
- after early returns that can change call order.

Wrong:

```
function Form({ showAdvanced }) {
  if (showAdvanced) {
    const [value, setValue] = useState("");
  }
  return null;
}
```

2.2 Why Hook Order Matters

React associates Hook state by call position, not by variable name.

Interview line:

- If Hook order changes between renders, React reads the wrong state bucket and the component becomes inconsistent.

2.3 Linting Support

`eslint-plugin-react-hooks` is not optional in serious codebases.

It helps enforce:

- Rules of Hooks,
- exhaustive dependencies,
- purity and modern React guidance.

3) Learn-Section Concepts That Interviewers Expect

3.1 Thinking in React

The standard decomposition approach:

1. Break UI into components.
2. Build static version first.
3. Find minimal but complete state.
4. Decide where state lives.
5. Add inverse data flow through callbacks or actions.

Interview line:

- The most common React design mistake is storing derived values in state instead of deriving them during render.

3.2 Choosing State Structure

Good state shape:

- minimal,
- normalized,
- avoids duplication,
- avoids contradictions.

Bad:

```
const [firstName, setFirstName] = useState("Ada");
const [lastName, setLastName] = useState("Lovelace");
const [fullName, setFullName] = useState("Ada Lovelace");
```

Better:

```
const fullName = `${firstName} ${lastName}`;
```

3.3 Queueing State Updates and Batching

React batches updates to reduce unnecessary renders.

Senior-level point:

- In React 18+, batching is broader and applies across more async boundaries than older React versions.

3.4 Preserving and Resetting State

State preservation depends on component identity:

- same component type in the same position usually preserves state,
- changing key or type resets state.

```
{isA ? <Profile key="a" /> : <Profile key="b" />}
```

Because the keys differ, React treats them as distinct identities and resets local state.

3.5 You Might Not Need an Effect

This is one of the most important react.dev interview topics.

Use an Effect only to synchronize with something outside React, such as:

- DOM APIs,
- subscriptions,
- timers,
- browser storage,
- third-party widgets,
- network or backend synchronization.

Do not use an Effect just to:

- derive filtered data,

- compute totals,
- mirror props into state without a real synchronization reason,
- run event-specific logic that belongs in the event handler.

Wrong:

```
const [fullName, setFullName] = useState("");

useEffect(() => {
  setFullName(`${firstName} ${lastName}`);
}, [firstName, lastName]);
```

Correct:

```
const fullName = `${firstName} ${lastName}`;
```

3.6 Separating Events from Effects

Event handlers answer:

- "What should happen because the user did something?"

Effects answer:

- "What should happen because this component is now on screen or synchronized to some external system?"

Mixing those two creates stale dependency lists and hard-to-debug reactivity problems.

4) Built-in Hooks by Category

4.1 Basic State Hooks

useState

Use when a component needs local state.

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount((current) => current + 1)}>
      {count}
    </button>
  );
}
```

Interview points:

- updates are scheduled, not immediate,
- updater functions avoid stale snapshots,
- keep state minimal.

useReducer

Use when state logic is complex, cross-field, or action-driven.

```
const initialState = { count: 0, error: null };

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + 1 };
    case "error":
      return { ...state, error: action.payload };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <button onClick={() => dispatch({ type: "increment" })}>
      {state.count}
    </button>
  );
}
```

When `useReducer` is better than `useState` :

- multiple transitions share logic,
- you want explicit event types,
- state updates depend on many fields,
- you want testable pure transition logic.

useActionState

`useActionState` is a newer hook for state that is updated through Actions, including async actions and server-driven form workflows.

```
function Checkout() {
  const [count, dispatchAction, isPending] = useActionState(
    async (previousCount, formData) => {
      const type = formData.get("type");
      return type === "ADD" ? previousCount + 1 : Math.max(0, previousCount - 1);
    },
    0
  );

  return (
    <form action={dispatchAction}>
      <p>Qty: {count}</p>
      <button name="type" value="ADD">Add</button>
    </form>
  );
}
```

```

    <button name="type" value="REMOVE">Remove</button>
    {isPending && <span>Saving...</span>}
  </form>
);
}

```

Interview points:

- designed for Actions and form workflows,
- can be sync or async,
- queues updates sequentially,
- especially relevant in React Server Components and progressive enhancement flows.

useOptimistic

`useOptimistic` lets you show optimistic UI while an Action is pending.

```

function LikeButton({ isLiked, saveLike }) {
  const [optimisticLiked, setOptimisticLiked] = useOptimistic(isLiked);

  function handleClick() {
    startTransition(async () => {
      setOptimisticLiked(!optimisticLiked);
      await saveLike(!optimisticLiked);
    });
  }

  return <button onClick={handleClick}>{optimisticLiked ? "Unlike" : "Like"}</button>;
}

```

Interview points:

- for latency hiding,
- often paired with `useActionState`, forms, or server mutations,
- use reducers when multiple optimistic fields must stay consistent.

4.2 Context Hooks

useContext

Use to read context from the nearest matching provider above the component.

```

const ThemeContext = createContext("light");

function Header() {
  const theme = useContext(ThemeContext);
  return <h1 className={theme}>Dashboard</h1>;
}

```

Interview points:

- avoids prop drilling,

- every consumer re-renders when the provider value identity changes,
- avoid passing freshly created objects unless intended.

use(resource) with Context

React's `use` API can also read context and is more flexible than `useContext` because it may appear inside conditionals and loops.

```
function Button({ show }) {
  if (!show) return null;
  const theme = use(ThemeContext);
  return <button className={theme}>Action</button>;
}
```

Interview nuance:

- `use` is not a normal Hook in terms of placement rules,
- but it still must be called from a component or custom Hook.

4.3 Ref Hooks

useRef

`useRef` stores a mutable value that survives renders without causing re-renders.

```
function TextInput() {
  const inputRef = useRef(null);

  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

Use cases:

- DOM access,
- storing timer IDs,
- storing previous values,
- mutable escape hatches.

useImperativeHandle

Use with `forwardRef` -style imperative exposure patterns to control what a parent can access.

```
const CustomInput = forwardRef(function CustomInput(_, ref) {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus() {
      inputRef.current.focus();
    }
  }));
});
```



```

    },
    clear() {
      inputRef.current.value = "";
    }
  }, []);

  return <input ref={inputRef} />;
});

```

Interview line:

- `useImperativeHandle` is an escape hatch. Prefer declarative props first, expose imperative APIs only for integration boundaries or focus/selection/scroll cases.

4.4 Effect Hooks

useEffect

Use `useEffect` to synchronize with external systems after paint.

```

function ChatRoom({ roomId }) {
  useEffect(() => {
    const connection = createConnection(roomId);
    connection.connect();
    return () => connection.disconnect();
  }, [roomId]);

  return null;
}

```

Interview points:

- cleanup runs before the next effect and on unmount,
- dependency arrays describe reactive inputs,
- async race conditions must be handled deliberately.

useLayoutEffect

Runs after DOM mutation but before browser paint.

Use cases:

- measuring layout,
- synchronously adjusting scroll or position,
- avoiding flicker for layout-sensitive reads/writes.

```

useLayoutEffect(() => {
  const rect = ref.current.getBoundingClientRect();
  setTooltipHeight(rect.height);
}, []);

```

Interview point:

- avoid by default because it blocks paint.

useInsertionEffect

A specialized hook for CSS-in-JS libraries to insert styles before layout effects run.

Interview line:

- application developers rarely need this directly; style engines use it to avoid layout measurement issues caused by late style insertion.

useEffectEvent

`useEffectEvent` lets you define Effect-only event logic that always sees the latest props/state without forcing the parent Effect to resubscribe.

```
function ChatRoom({ roomId, muted }) {
  const onConnected = useEffectEvent(() => {
    if (!muted) {
      showNotification("Connected");
    }
  });

  useEffect(() => {
    const connection = createConnection(roomId);
    connection.on("connected", onConnected);
    connection.connect();
    return () => connection.disconnect();
  }, [roomId]);
}
```

Interview points:

- solves a real stale-closure problem,
- not a dependency-array cheat code,
- can only be called from Effects or other Effect Events.

4.5 Performance Hooks

useMemo

Memoizes a computed value.

```
const filteredUsers = useMemo(() => {
  return users.filter((user) => user.name.includes(query));
}, [users, query]);
```

Use when:

- the computation is measurably expensive,
- or referential stability is needed for memoized children.

Do not use as cargo-cult optimization.

useCallback

Memoizes a function identity.

```
const handleSelect = useCallback((id) => {  
  setSelectedId(id);  
}, []);
```

Interview points:

- only useful if a stable identity matters,
- common with `React.memo`, effect dependencies, or third-party subscriptions.

useTransition

Marks updates as non-urgent so urgent UI can stay responsive.

```
function Search({ items }) {  
  const [query, setQuery] = useState("");  
  const [results, setResults] = useState(items);  
  const [isPending, startTransition] = useTransition();  
  
  function handleChange(e) {  
    const nextQuery = e.target.value;  
    setQuery(nextQuery);  
  
    startTransition(() => {  
      setResults(expensiveSearch(items, nextQuery));  
    });  
  }  
  
  return (  
    <>  
      <input value={query} onChange={handleChange} />  
      {isPending && <div>Updating...</div>}  
    </>  
  );  
}
```

Interview line:

- urgent updates keep the input responsive; non-urgent work may be interrupted.

useDeferredValue

`useDeferredValue` lets a slower part of the tree lag behind a fast-changing value.

```
function Search({ items }) {  
  const [query, setQuery] = useState("");  
  const deferredQuery = useDeferredValue(query);  
  const filtered = useMemo(  
    () => items.filter((item) => item.includes(deferredQuery)),
```

```

    [items, deferredQuery]
  );

  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <ResultsList items={filtered} />
    </>
  );
}

```

Interview point:

- the input updates immediately, while the heavy subtree renders from the deferred value.

4.6 External System Hooks

useSyncExternalStore

This is the correct React API for subscribing to external mutable stores in a concurrent-safe way.

```

function useOnlineStatus() {
  return useSyncExternalStore(
    (callback) => {
      window.addEventListener("online", callback);
      window.addEventListener("offline", callback);
      return () => {
        window.removeEventListener("online", callback);
        window.removeEventListener("offline", callback);
      };
    },
    () => navigator.onLine
  );
}

```

Interview points:

- preferred for external stores over ad hoc `useEffect + useState`,
- prevents tearing and inconsistent concurrent reads,
- commonly used by libraries like Redux bindings.

useDebugValue

Used inside custom Hooks to label values in React DevTools.

```

function useFriendStatus(friendId) {
  const isOnline = useOnlineStatus(friendId);
  useDebugValue(isOnline ? "Online" : "Offline");
  return isOnline;
}

```

Interview line:

- mostly for custom Hook ergonomics in shared libraries.

4.7 Other Built-in Hooks and APIs

useId

Used for stable unique IDs that work across server rendering and hydration.

```
function EmailField() {
  const id = useId();
  return (
    <>
      <label htmlFor={` ${id}-email`} >Email</label>
      <input id={` ${id}-email`} />
    </>
  );
}
```

use

`use(resource)` reads a Promise or context value.

```
function Message({ messagePromise }) {
  const message = use(messagePromise);
  return <p>{message}</p>;
}
```

Interview points:

- integrates with Suspense and Error Boundaries,
- especially relevant to Server Components,
- when used with Promises, rejected values are handled via Error Boundaries or `Promise.catch`.

5) Custom Hooks

Custom Hooks are not a special runtime feature. They are ordinary functions that:

- call Hooks,
- package reusable stateful logic,
- hide implementation details.

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);
}
```

```
    return width;
  }
```

Interview expectation:

- extract by behavior, not by JSX,
- custom Hooks improve reuse and testability,
- they still obey Rules of Hooks.

6) Reconciliation and State Identity

6.1 What Reconciliation Actually Means

React compares the previous tree and next tree, then applies the minimum necessary commits.

High-level rules:

- same component type in same position: update existing instance,
- different type: tear down old subtree and mount new one,
- keys guide identity for siblings.

6.2 The `key` Prop Is About Identity, Not Just Warnings

Wrong:

```
{items.map((item, index) => (  
  <Row key={index} item={item} />  
))}
```

Why this is risky:

- reorder/filter operations can attach the wrong local state to the wrong row,
- inputs can "jump",
- animations and focus become buggy.

Correct:

```
{items.map((item) => (  
  <Row key={item.id} item={item} />  
))}
```

6.3 Preserving vs Resetting State in Lists

Interview line:

- keys tell React whether something is the same thing over time.

If two siblings swap positions but keep stable keys, their state moves with them. If keys change, state resets.

7) React DOM APIs You Should Know

7.1 Client APIs

createRoot

Modern client entry point.

```
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<App />);
```

hydrateRoot

Used when HTML was pre-rendered on the server and React must attach behavior without replacing the DOM.

Interview point:

- hydration mismatches happen when server and client render different output.

7.2 Portal and Flush APIs

createPortal

Renders children into a different DOM subtree while preserving the React tree relationship.

```
return createPortal(<Modal />, document.body);
```

Use cases:

- modals,
- tooltips,
- popovers,
- overlays escaping stacking contexts.

flushSync

Forces React to flush updates synchronously.

Interview line:

- use rarely, mostly for integration with imperative browser or third-party APIs that require DOM to be updated immediately.

7.3 Resource Hint APIs

Modern `react-dom` also includes resource hint helpers such as:

- `preconnect` ,
- `prefetchDNS` ,
- `preload` ,
- `preloadModule` ,
- `preinit` ,
- `preinitModule` .

Why interviewers care:

- these help reduce latency for scripts, styles, fonts, images, and module graphs,

- they matter in streaming SSR and framework-controlled performance pipelines.

Interview framing:

- `preconnect` opens the connection early,
- `prefetchDNS` resolves DNS early,
- `preload` fetches a critical resource early,
- `preinit` is for eagerly preparing resources like scripts/styles sooner in the pipeline.

7.4 Server and Static APIs

Important server-side APIs to recognize:

- `renderToPipeableStream` for Node streaming,
- `renderToReadableStream` for Web Streams runtimes,
- static prerender/resume family in newer React server pipelines.

Interview point:

- modern frameworks use streaming SSR rather than waiting for the whole tree before sending HTML.

8) Suspense, Lazy Loading, and Async Boundaries

8.1 React.lazy

```
const HeavyChart = lazy(() => import("./HeavyChart"));

function Dashboard() {
  return (
    <Suspense fallback={<Spinner />}>
      <HeavyChart />
    </Suspense>
  );
}
```

8.2 Suspense

Suspense lets React coordinate loading boundaries for:

- lazy component code,
- Promise-based async data with `use`,
- framework integrations.

Interview line:

- Suspense is not a generic data-fetching library by itself; it is a coordination primitive that frameworks and compatible data layers can integrate with.

9) React Server Components and Boundaries

9.1 Server Components vs Client Components

Server Components:

- run on the server,

- can access secrets and databases,
- reduce client bundle size,
- cannot use client-only hooks like `useState` .

Client Components:

- handle interactivity,
- use state/effects/browser APIs,
- require client JavaScript.

Client marker:

```
"use client";

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount((c) => c + 1)}>{count}</button>;
}
```

9.2 Streaming Promises to Clients

An important React 19-era pattern:

- create Promises on the server,
- pass them down,
- read them with `use` in a client boundary wrapped by `Suspense`.

9.3 Forms, Actions, and Progressive Enhancement

With newer React patterns:

- forms can submit through Actions,
- `useActionState` and `useOptimistic` improve server mutation UX,
- progressive enhancement can preserve usability before full hydration.

10) Performance Optimization Strategy

10.1 Optimize at the Right Layer

Senior answer structure:

1. Measure first.
2. Find whether the bottleneck is network, bundle, render cost, layout, or data churn.
3. Optimize the narrowest real bottleneck.

10.2 React.memo

```
const Row = React.memo(function Row({ item, onSelect }) {
  return <li onClick={() => onSelect(item.id)}>{item.name}</li>;
});
```

Useful when:

- parent re-renders often,

- child props are stable,
- child rendering is not trivial.

10.3 Virtualization

For very large lists, render only visible rows.

```
import { FixedSizeList } from "react-window";

function VirtualList({ items }) {
  return (
    <FixedSizeList height={500} itemCount={items.length} itemSize={36} width={600}>
      {( { index, style } ) => <div style={style}>{items[index]}</div>}
    </FixedSizeList>
  );
}
```

10.4 Bundle-Level Optimization

- split by route and heavy feature,
- prefer Server Components where appropriate,
- avoid shipping server-only dependencies,
- defer non-critical UI,
- use framework image/font/script optimization.

10.5 React Compiler

react.dev now documents the React Compiler direction.

Interview framing:

- the compiler aims to automate many memoization patterns,
- this reduces the need for manual `useMemo` / `useCallback` in some cases,
- but teams still need correct mental models, purity, and safe code patterns.

10.6 Profiling

Use:

- React DevTools Profiler,
- browser Performance panel,
- Web Vitals,
- flamecharts,
- framework bundle analyzers.

11) Core Web Vitals and UX Metrics

11.1 Main Metrics

- LCP: largest contentful paint. Good target: under 2.5s.
- INP: interaction to next paint. Good target: under 200ms.
- CLS: cumulative layout shift. Good target: under 0.1.

Interview point:

- FID is older interview vocabulary; modern discussions increasingly use INP.

11.2 Measuring

```
import { onCLS, onINP, onLCP } from "web-vitals";

onCLS(console.log);
onINP(console.log);
onLCP(console.log);
```

12) Strict Mode, Common Pitfalls, and Interview Traps

12.1 Why Effects May Run Twice in Development

Strict Mode intentionally replays some logic in development to surface impure behavior and missing cleanup.

Interview line:

- if your Effect cannot tolerate setup-cleanup-setup in development, it is usually a sign the synchronization logic is incomplete.

12.2 Common Hook Mistakes

- storing derived data in state,
- missing effect dependencies,
- using Effects for user events,
- mutating state objects in place,
- using array indexes as keys,
- overusing `useMemo` and `useCallback` ,
- using refs as hidden state instead of modeling real UI state.

12.3 When to Reach for Escape Hatches

Escape hatches include:

- refs,
- imperative handles,
- layout effects,
- insertion effects,
- `flushSync`.

Interview line:

- these are valid tools, but they are signs you are leaving the normal declarative path. Use them precisely and sparingly.

13) Class Components and Error Boundaries

13.1 Why Class Components Still Matter in Interviews

- legacy codebases still have them,
- Error Boundaries are still commonly implemented as class components,
- migration knowledge is often tested.

13.2 Lifecycle Mapping

- `componentDidMount` roughly maps to mount synchronization.
- `componentDidUpdate` maps to update synchronization.
- `componentWillUnmount` maps to cleanup.

13.3 Error Boundary Example

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error(error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

Interview point:

- Error Boundaries catch rendering, lifecycle, and constructor errors in descendant trees, not event handler errors.

14) Senior-Level Revision Checklist

- ☐ Explain render vs commit without hand-waving.
- ☐ Explain why render must stay pure.
- ☐ Explain when you do not need an Effect.
- ☐ Explain `useState` , `useReducer` , `useContext` , and `useRef` with correct trade-offs.
- ☐ Explain `useTransition` vs `useDeferredValue` .
- ☐ Explain `useActionState` , `useOptimistic` , and `use` in modern React.
- ☐ Explain `useSyncExternalStore` and why external stores need a dedicated integration API.
- ☐ Explain `useEffectEvent` without using it as a dependency-array hack.
- ☐ Explain state preservation and reset using keys and identity.
- ☐ Explain Suspense, lazy loading, and server/client boundaries.
- ☐ Explain `createPortal` , hydration, and at least one streaming server API.
- ☐ Explain when `React.memo` helps and when it is noise.
- ☐ Explain Core Web Vitals with modern terminology.
- ☐ Explain where the React Compiler changes the memoization conversation.